

模块化的力量

函数、递归与库的威力 · 12题 · C++ & Python 双语对照

小鲁已经写了上百行代码了。但他发现一个问题："同样的逻辑我写了好几遍——求最大值、算阶乘、交换变量.....能不能写一次，到处用？"小华笑道："你终于感受到**函数**的需求了！函数是编程中最基本、最重要的抽象手段——把一段逻辑打包装箱，贴个标签，以后直接叫名字就能用。"

本章12道题分三个板块：**函数定义与调用**（NQ083-088）——学会参数传递、返回值、数组传参；**递归思维**（NQ089-093）——函数自己调用自己，基础案例→记忆化优化→回溯搜索；**标准库的威力**——Python的math/lib等内置库让你的代码量减少90%。记住：**写函数的第一原则是**——先看看标准库有没有现成的。

本章角色

-  **小鲁**（初学者）· 从"重复代码"的痛苦中领悟函数的必要性
-  **小华**（算法高手）· 深入讲解C++引用传递vs Python对象引用，递归vs递推
-  **小栋**（工程师）· 用网格走方格、跳台阶等实际问题引入DP思维
-  **小嘉**（校主精神）· 带来《九章算术》的gcd算法，强调数学与编程的千年共鸣

函数三大价值 + C++ vs Python 对比

- **封装**：把逻辑打包→隐藏细节→只暴露接口。调用者不需要知道内部实现
- **复用**：写一次用百次。修改时只改一处
- **抽象**：fact(n)比"从1乘到n"高一个思考层次——用函数名表达意图
- **参数传递**：C++值传递vs引用传递(&)→&使函数能修改外部变量；Python"传对象引用"→可变对象(list)可被修改，不可变对象(int)不能
- **递归vs递推**：递归自顶向下（优雅但耗栈），递推自底向上（高效但需手动管理状态）
- **标准库优先**：math.gcd/factorial/comb, itertools.permutations, @lru_cache——先查库再动手

NQ083 n的阶乘 AcWing 804

小鲁正对着一道数学题发愁："5! = 5×4×3×2×1 = 120，这个我还能手算.....但是如果要算10!怎么办？"小华拉过键盘："函数就是做这个的！你把'计算阶乘'封装成一个fact(n)函数——每次需要算阶乘的时候调用它就行。这就叫'封装'。Python直接import math; math.factorial(10)一行搞定，但理解函数定义与调用的关系，比直接用库更重要。"

- 输入** 共一行，包含一个整数n。
- 输出** 共一行，包含一个整数表示n的阶乘的值。
- 范围** $1 \leq n \leq 10$

输入

输出

思路 阶乘 $n! = 1 \times 2 \times \dots \times n$ 。用for循环累乘。C++需定义`int fact(int n)`函数，Python可以`def fact(n)`或直接用`math.factorial()`。int在 $n \leq 10$ 范围内安全（最大 $10! = 3,628,800$ ）。

技巧 Python的`math.factorial()`使用高效的C实现。手写版关键是用`result=1`初始化然后`for i in range(1,n+1):result*=i`。初始化是初学者最常犯的bug——用0而不是1会导致结果永远是0。

C++

```
#include <iostream>

using namespace std;

int fact(int n)
{
    int res = 1;
    for (int i = 1; i <= n; i ++ )
        res *= i;
    return res;
}

int main()
{
    int n;
    cin >> n;

    cout << fact(n) << endl;

    return 0;
}
```

Python

```
import math;print(math.factorial(int(
input())))
```

NQ084 x和y的最大值 AcWing 805

"写个函数，输入两个数，返回较大的那个。"小栋对小鲁说。"这还用函数？直接if判断不就行了。"小鲁反驳。小华摇摇头："在大的程序里，你可能要在几十个地方都用max——每次都写if多麻烦，而且如果max逻辑改了（比如要支持更多参数），你要改几十处。有了`max(x,y)`函数，改一次，处处生效。这就是函数的第二个价值：代码复用。"

输入 一行，两个整数x和y。

输出 一个整数，较大者。

范围 $-100 \leq x, y \leq 100$

输入

3 5

输出

5

思路 函数封装max逻辑：if $x > y$ return x else return y 。C++用`int max(int x,int y)`，Python用`def my_max(x,y): return x if x>y else y`。理解函数签名（参数名+类型+返回值）是设计函数的第一步。

技巧 C++的三元运算符: `return x>y?x:y`。Python: `return x if x>y else y`。两种语法看似不同, 语义完全一样——条件→值1→值2。

C++

```
#include <iostream>

using namespace std;

int max(int x, int y)
{
    if (x > y) return x;
    return y;
}

int main()
{
    int x, y;
    cin >> x >> y;

    cout << max(x, y) << endl;

    return 0;
}
```

Python

```
x,y=map(int,input().split());print(max(x,y))
```

NQ085 最大公约数 AcWing 808

小嘉从南洋经商归来, 带回了一道古老的问题: "两个正整数, 找到能同时整除它们的最大数——这就是《九章算术》里的'更相减损术', 也就是欧几里得算法。"小华在屏幕上写道: "`gcd(a,b)=gcd(b,a%b)`", 一直递归到 `b=0`时返回`a`。这是两千年前的算法, 至今仍然是求GCD最高效的方法。Python的`math.gcd()`内部用的就是它。"

- 输入** 一行, 两个整数`a`和`b`。
- 输出** 一个整数, `a`和`b`的最大公约数。
- 范围** $1 \leq a, b \leq 10^9$

输入	输出
12 18	6

思路 欧几里得算法: `while b: a,b = b,a%b; return a`。Python的`a,b=b,a%b`一行完成交换和取余。C++用 `while(b){int t=a%b;a=b;b=t;}`。时间复杂度 $O(\log \min(a,b))$ ——对 10^9 只需约30次运算。

技巧 Python一行gcd: `while b: a,b = b, a%b`。这个"元组解包+赋值"的写法是Python递推算法的标志性代码。`math.gcd()`底层是C实现, 更快但手写更理解原理。

C++

```
#include <iostream>

using namespace std;

int gcd(int a, int b)
{
    for (int i = 1000; i; i -- )
        if (a % i == 0 && b % i == 0)
            return i;
    return -1;
}

int main()
{
    int a, b;
    cin >> a >> b;
    cout << gcd(a, b) << endl;

    return 0;
}
```

Python

```
import math;a,b=map(int,input().split())
print(math.gcd(a,b))
```

NQ086 交换数值 AcWing 811

"交换两个变量的值——这是编程的'hello world'动作。"小华说，"C++需要传引用（&）或者用指针，否则函数内的交换不会影响外部变量。Python的函数参数是'传对象引用'——列表可变会受影响，整数不可变不会。但Python有更优雅的方式：a,b=b,a——直接元组解包交换，不需要函数！"

输入

一行，两个整数x和y。

输出

一行，交换后的两个整数（用空格分开）。

范围

$-10^9 \leq x, y \leq 10^9$

输入

3 5

输出

5 3

思路 C++的难点：值传递vs引用传递。void swap(int &a,int &b){int t=a;a=b;b=t;}中的&是关键——没有&则函数内的交换不影响调用处。Python直接用x,y=y,x一行交换，无需定义函数。理解这个差异是理解两种语言参数传递模型的关键。

技巧 Python的a,b=b,a是元组解包——等式右边先创建元组(b,a)，然后解包赋值给a和b。这是Python最优雅的语法糖之一。C++用std::swap(a,b)最简单。

C++

```
#include <iostream>

using namespace std;

void swap(int& x, int& y)
{
    if (x == y) return;

    int t = x;
    x = y;
    y = t;
}

int main()
{
    int x, y;
    cin >> x >> y;
    swap(x, y);

    cout << x << ' ' << y << endl;

    return 0;
}
```

Python

```
a,b=map(int,input().split());a,b=b,a;
print(a,b)
```

NQ087 打印数字 AcWing 812

"我想写一个函数print_array(arr, size)，传入一个数组和它的大小，自动打印所有元素。"小鲁跃跃欲试。小华说："C++中数组作为参数时退化为指针，必须显式传size。Python的list自带长度信息，不需要额外参数。这个差异体现了两种语言的设计哲学：C追求零开销，Python追求方便安全。"

输入 第一行n和size，第二行n个数。

输出 打印前size个元素，空格分隔。

范围 $1 \leq \text{size} \leq n \leq 1000$

输入

5 3
1 2 3 4 5

输出

1 2 3

思路 C++数组参数退化为指针——void print(int a[], int size)。必须传size。Python的list自带__len__，for x in arr自动遍历。数组传参是理解C指针与Python引用差异的典型示例。

技巧 C++中int a[]作参数等同于int* a——丢失了长度信息。这就是为什么C风格的数组操作几乎都带size参数。Python没有这个问题——每个list都知道自己的长度。

C++

```
#include <iostream>

using namespace std;

const int N = 1010;

void print(int a[], int size)
{
    for (int i = 0; i < size; i ++ )
        cout << a[i] << ' ';
    cout << endl;
}

int main()
{
    int n, size;
    int a[N];

    cin >> n >> size;
    for (int i = 0; i < n; i ++ ) cin >> a[i];

    print(a, size);

    return 0;
}
```

Python

```
n=int(input());arr=list(map(int,input().split()))
for x in arr:print(x)
```

NQ088 打印矩阵 AcWing 813

"升级了！现在要打印一个二维矩阵。"小栋把题目升级。小华说："C++需要指定第二维的大小——void print(int a[][100], int row, int col)。注意：只有第一维可以省略，第二维必须指定（编译器需要知道每行的大小来做地址计算）。Python就简单多了——list of lists，遍历即打印。"

输入 第一行row和col，然后row行col列。

输出 按行列格式打印矩阵。

范围 $1 \leq \text{row}, \text{col} \leq 100$

输入	输出
2 3	1 2 3
1 2 3	4 5 6
4 5 6	

思路 C++二维数组参数：void print(int a[][100], int r, int c)。第二维必须指定大小。Python: def print_matrix(mat): for row in mat: print(*row)——键解包输出。

技巧 Python的print(*row)用*解包列表——等价于print(row[0],row[1],row[2])。这是Python打印列表最优雅的写法。C++的二维数组参数中第二维大小是编译器常量限制——动态大小的矩阵需要用vector或动态数组。

C++

```
#include <iostream>

using namespace std;

void print2D(int a[][100], int row, int col)
{
    for (int i = 0; i < row; i ++ )
    {
        for (int j = 0; j < col; j ++ )
            cout << a[i][j] << ' ';
        cout << endl;
    }
}

int main()
{
    int a[100][100];

    int row, col;

    cin >> row >> col;
    for (int i = 0; i < row; i ++ )
        for (int j = 0; j < col; j ++ )
            cin >> a[i][j];

    print2D(a, row, col);

    return 0;
}
```

Python

```
r,c=map(int,input().split())
for _ in range(r):print(' '.join(input().split()))
```

NQ089 递归求阶乘 AcWing 819

"之前我们用循环写阶乘——现在用递归！"小华说，"递归就是函数自己调用自己。fact(n)=n×fact(n-1)，fact(0)=1。代码只有3行，但理解它需要'递归思维'——假设fact(n-1)已知，你只需要乘上n。"小鲁试着在纸上展开fact(4)→4×fact(3)→4×3×fact(2)→...→24。"确实优雅！但感觉比循环多用了空间。"

输入 一个整数n。

输出 n的阶乘。

范围 $0 \leq n \leq 10$

输入

4

输出

24

思路 递归三板斧：1.基条件if n==0 return 1；2.递归调用fact(n-1)；3.组合结果n*fact(n-1)。递归需要调用栈——每层递归占用栈空间。循环O(1)空间，递归O(n)栈空间——这是递归的代价。

技巧 Python递归深度默认限制1000层（可改）。递归优雅但对深度大的问题可能栈溢出（Stack Overflow——是的，那个著名的程序员问答网站名字就这么来的）。小n用递归，大n用循环。

C++

```
#include <iostream>

using namespace std;

int fact(int n)
{
    if (n == 1) return 1;
    return n * fact(n - 1);
}

int main()
{
    int n;
    cin >> n;
    cout << fact(n) << endl;

    return 0;
}
```

Python

```
def f(n): return 1 if n<=1 else n*f(n-1)
print(f(int(input())))
```

NQ090 递归求斐波那契数列 AcWing 820

" $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ ——多优雅的递归定义！"小鲁兴奋地写下了递归版的斐波那契。"但是 $n=40$ 时我的电脑卡住了！"小华严肃地说："这就是经典的反面教材——朴素的递归斐波那契做了大量重复计算（ $\text{fib}(5)$ 被算了8次！）。解决方法有两种：循环递推或带缓存的递归。Python的@lru_cache装饰器——一行注解，递归变成 $O(n)$ ，魔法般的效果。"

输入

一个整数 n 。

输出

第 n 项斐波那契数。

范围

 $0 \leq n \leq 60$

输入

5

输出

5

思路 朴素递归 $O(2^n)$ ——指数爆炸。加缓存（Python的@lru_cache或手写字典记录已算过的值） $\rightarrow O(n)$ 。递推循环 $O(n)+O(1)$ 空间。递归+缓存和递推循环等价——前者自顶向下（Top-down DP），后者自底向上（Bottom-up DP）。

技巧 Python: from functools import lru_cache; @lru_cache(None); def fib(n): return n if $n < 2$ else $\text{fib}(n-1) + \text{fib}(n-2)$ 。@lru_cache是Python动态规划的核武器——加一行注解，任何纯函数的重复计算问题瞬间解决。

C++

```
#include <iostream>

using namespace std;

int f(int n)
{
    if (n <= 2) return 1;
    return f(n - 2) + f(n - 1);
}

int main()
{
    int n;
    cin >> n;
    cout << f(n) << endl;

    return 0;
}
```

Python

```
def f(n):return n if n<=1 else f(n-1)
+f(n-2)
print(f(int(input())))
```

NQ091 跳台阶 AcWing 821

小鲁在操场上练习跳台阶——每次可以跳1级或2级。"10级台阶有多少种不同的跳法？"小华问。小鲁想了想："到第n级的跳法=到第n-1级的跳法（跳1级上来）+到第n-2级的跳法（跳2级上来）。这不就是斐波那契数列吗？"小华笑了："没错！这就是为什么递推思维如此重要——表面上不同的问题，底层可能是同一个数学模型。"

输入 一个整数n。

输出 跳法总数。

范围 $1 \leq n \leq 15$

输入

4

输出

5

思路 $f(n)=f(n-1)+f(n-2)$, $f(1)=1, f(2)=2$ 。和斐波那契的递推公式一样但基条件不同（ $f(2)=2$ 而非1）。注意 $n \leq 15$ 时int足够。Python一行：a,b=1,2;for _ in range(n-1):a,b=b,a+b;print(a)。

技巧 这是最简单的DP入门题。学会识别"可以从前一步/前两步转移过来"的模式。这种"状态转移"思维将贯穿整个算法学习——背包、最长子序列、编辑距离……本质都是这个框架。

C++

```
#include <iostream>

using namespace std;

int n;
int ans;

void f(int k)
{
    if (k == n) ans ++ ;
    else if (k < n)
    {
        f(k + 1);
        f(k + 2);
    }
}

int main()
{
    cin >> n;
    f(0);
    cout << ans << endl;

    return 0;
}
```

Python

```
from functools import lru_cache
@lru_cache(None)
def f(n):return n if n<=2 else f(n-1)
+f(n-2)
n=int(input());print(f(n))
```

NQ092 走方格 AcWing 822

小栋画了一个 $n \times m$ 的网格：从 $(0,0)$ 走到 (n,m) ，每次只能向右或向下走一步——有多少种不同的走法？"小鲁兴奋地说："这不就是组合数 $C(n+m, n)$ 吗？总共要走 $n+m$ 步，选其中 n 步向右。"小华补充道："对——但如果你想用代码算，递推公式 $f[i][j]=f[i-1][j]+f[i][j-1]$ 更通用。Python一行`math.comb(n+m, n)`搞定——标准库的强大之处。"

输入

一行，两个整数 n 和 m 。

输出

走法总数。

范围

 $1 \leq n, m \leq 10$

输入

2 3

输出

10

思路 组合数 $C(n+m, n) = (n+m)! / (n! \times m!)$ 。Python: `math.comb(n+m, n)`一行。递推DP: $f[i][j]=f[i-1][j]+f[i][j-1]$; $f[0][*]=f[*][0]=1$ 。两种思维都正确——前者用数学知识，后者用DP框架。

技巧 Python 3.8+的`math.comb(n,k)`计算组合数——比手写阶乘除法更安全（避免了整数溢出）。C++中 $n+m \leq 20$ 时可用long存组合数。

C++

```
#include <iostream>

using namespace std;

int n, m;
int ans;

void dfs(int x, int y)
{
    if (x == n && y == m) ans ++ ;
    else
    {
        if (y < m) dfs(x, y + 1);
        if (x < n) dfs(x + 1, y);
    }
}

int main()
{
    cin >> n >> m;
    dfs(0, 0);
    cout << ans << endl;

    return 0;
}
```

Python

```
import math;n,m=map(int,input().split())
print(math.comb(n+m,n))
```

NQ093 排列 AcWing 823

"用递归生成1到n的所有排列——这是搜索算法的入门课。"小华在白板上画出搜索树："第1层选第1个数（n种选择），第2层选第2个数（n-1种选择）……一直到选完n个数。每次递归深入一层，用used数组标记哪些数已被用了。这个回溯框架将贯穿你的整个搜索算法学习——从排列到N皇后，从数独到旅行商问题。"

输入 一个整数n。

输出 1~n的所有排列，每行一个排列。

范围 $1 \leq n \leq 9$

输入	输出
3	1 2 3 1 3 2 2 1 3 2 3 1 3 1 2 3 2 1

思路 DFS回溯：dfs(path, used)，当len(path)==n时输出。每次从未被used标记的数中选一个加入path，递归完成后撤销选择（回溯）。Python的itertools.permutations(range(1,n+1))一行搞定——库函数也是用类似的算法实现的。

技巧 Python: `from itertools import permutations; for p in permutations(range(1,n+1)): print(*p)`。手写回溯用递归+used数组，itertools用C实现更快。理解回溯框架>会调库——因为库不能解决所有搜索问题。

C++

```
#include <iostream>

using namespace std;

const int N = 10;

int n;

void dfs(int u, int nums[], bool st[])
{
    if (u > n)
    {
        for (int i = 1; i <= n; i ++ ) cout << nums[i] << ' ';
        cout << endl;
    }
    else
    {
        for (int i = 1; i <= n; i ++ )
            if (!st[i])
            {
                st[i] = true;
                nums[u] = i;
                dfs(u + 1, nums, st);
                st[i] = false;
            }
    }
}

int main()
{
    cin >> n;

    int nums[N];
    bool st[N] = {0};

    dfs(1, nums, st);

    return 0;
}
```

Python

```
from itertools import permutations
n=int(input())
for p in permutations(range(1,n+1)):p
rint(*p)
```

NQ094 数组排序 AcWing 818

"这一章的最后一道题——写一个函数把数组排序。"小华说，"当然Python有`sort()`，C++有`std::sort()`，但手写排序算法的意义在于——理解不同排序的思想：选择排序每次选最小的放前面，冒泡排序相邻比较交换，插入排序像整理扑克牌……这些思维训练让你的算法直觉不断提升。"

输入 第一行`n`和`size`，第二行`n`个数。

输出 排序后的前`size`个元素。

范围 $1 \leq \text{size} \leq n \leq 1000$

输入

5 3
3 1 4 1 5

输出

1 1 3

思路 多种排序可选：选择排序（找最小放前面）、冒泡排序（相邻交换）、插入排序（扑克牌式）。复杂度 $O(n^2)$ 对 $n \leq 1000$ 足矣。Python: `arr.sort()`或`sorted(arr)`一行。C++: `sort(arr, arr+n)`使用快速排序 $O(n \log n)$ 。

技巧 Python的`sort()`是Timsort（归并+插入的混合算法），C++的`std::sort()`是Introsort（快速排序+堆排序的混合）。标准库的排序经过数十年优化——实际开发中绝不要手写排序。

C++

```
#include <iostream>

using namespace std;

void sort(int a[], int l, int r)
{
    for (int i = l; i <= r; i++)
        for (int j = i + 1; j <= r; j++)
            if (a[j] < a[i])
                swap(a[i], a[j]);
}

int main()
{
    int a[1000];
    int n, l, r;
    cin >> n >> l >> r;
    for (int i = 0; i < n; i++) cin >> a[i];

    sort(a, l, r);

    for (int i = 0; i < n; i++) cout << a[i] << ' ';

    return 0;
}
```

Python

```
n=int(input());arr=list(map(int,input().split()));arr.sort();print(*arr)
```

本章知识点总结

- 函数三价值：封装（隐藏细节）、复用（一次编写多次调用）、抽象（更高层次的思考）
- 参数传递差异：C++值vs引用(&)，Python“传对象引用”——理解这个差异是写出正确函数的前提
- 递归三板斧：基条件→递归调用→组合结果。注意栈溢出风险和重复计算问题
- @lru_cache：一行注解将指数级递归变成多项式级——Python动态规划的核武器
- 回溯框架：选→递归→撤销。排列问题是搜索算法的入门课，这个框架贯穿整个算法学习
- 标准库优先：math.gcd/factorial/comb, itertools.permutations, functools.lru_cache——先查库再动手